DATABRICKS

Interview Preparation Guide

Comprehensive Style · Interview-Essential Topics Only · Quick Revision

March 19, 2026

1. Introduction to Databricks & Apache Spark

1.1 What is Databricks?

Databricks is a cloud-based unified analytics platform built on top of Apache Spark. Founded by the original creators of Apache Spark, it removes the complexity of cluster infrastructure by providing a fully managed, collaborative workspace where data engineers, data scientists, and analysts can focus entirely on data — not on managing servers.

Databricks runs on all three major clouds (Azure, AWS, GCP) and continuously evolves with platform features such as Delta Lake (transactional storage), Unity Catalog (unified governance), Workflows (job orchestration), and Lakeflow (next-generation data engineering).

Why Databricks?	Explanation
Simplifies Apache Spark	Removes cluster setup and management complexity — teams focus on data and analytics.
Collaborative Workspace	Multi-language notebooks (Python, SQL, Scala, R), dashboards, and REST APIs for joint development.
Cloud Agnostic	Runs on Azure, AWS, and GCP. Azure Databricks integrates tightly with AAD, ADLS Gen2, ADF, and Key Vault.
Delta Lake Built-in	ACID transactions, schema enforcement, and time travel on cloud object storage — out of the box.
Unity Catalog	Centralised governance across all workspaces — one policy, one audit log, one lineage graph.
Industry Demand	Databricks + Spark skills are among the most in-demand in data engineering today.

1.2 The Two-Plane Architecture

Databricks infrastructure is split into two planes. Understanding this separation is important for architecture questions, security discussions, and understanding where code runs vs where it is managed.

Plane	Responsibilities	Owner
Control Plane	Web UI, REST APIs, notebook authoring, cluster & job definitions, access control, governance, metadata management, job scheduling	Databricks-managed cloud infrastructure
Compute Plane (Data Plane)	Driver + worker VMs, actual Spark code execution, cloud object storage (ADLS/S3/GCS), VNet/VPC networking, dynamically provisioned compute resources	Customer's own cloud account (VPC/VNet)

Key Interview Point

The customer's data never leaves their cloud account — it lives in the Compute Plane which is inside the customer's own VPC/VNet. The Control Plane only manages metadata and orchestration. This separation is the foundation of Databricks' security model.

1.3 Spark Architecture

Apache Spark is the distributed computing engine at the core of Databricks. A Spark application runs as coordinated processes: one Driver that orchestrates, and many Executors on Worker Nodes that process data in parallel.

Component	Role	Critical Detail
Cluster Manager	Allocates resources and launches the Spark application	Abstracted in Databricks — managed automatically
Driver	Parses code, builds the DAG, schedules tasks, orchestrates execution	Does NOT process data partitions. Single node — never collect large data to it.
Executors	Execute tasks on data partitions, manage cache, report results to Driver	One executor per worker node in Databricks
Worker Nodes	Host executors and perform actual computation	Scale horizontally — add nodes to increase throughput

1.4 Lazy Evaluation & Transformations vs Actions

Lazy evaluation is one of Spark's most important design principles. When you write a transformation (filter, select, join), Spark does not execute it immediately — it records it in a Directed Acyclic Graph (DAG). Execution happens only when an Action is called. This allows the Catalyst Optimizer to inspect the full plan and apply optimisations before any data is read.

Concept	Definition	Examples
Transformation (Lazy)	Defines a new DataFrame without executing. Builds the DAG only.	filter, select, join, withColumn, groupBy, map
Narrow Transformation	One input partition → one output partition. No data shuffle needed. Fast.	filter, map, withColumn, union
Wide Transformation	Requires shuffling data across partitions. Expensive network I/O. Creates stage boundaries.	groupBy, sort, join, distinct, repartition
Action	Triggers actual computation of the DAG. Returns a result or writes data.	count(), collect(), show(), write(), save()

Language API	Notes
Python (PySpark)	Most widely used. DataFrames and MLlib. Some Python UDF overhead.
SQL (Spark SQL)	ANSI SQL 2003. Optimised by Catalyst. No performance difference vs DataFrame API.
Scala	Native Spark language. JVM-native, lowest overhead.
R (SparkR)	Integration for R ecosystem.

Key Interview Point

All language APIs — SQL, DataFrame, Dataset — compile to the same logical and physical execution plan via the Catalyst Optimizer. There is NO performance difference between writing a query in Python, Scala, or SQL. The main entry point is SparkSession (available as 'spark' in all Databricks notebooks).

2. Notebooks, DBFS & Databricks Utilities

2.1 Magic Commands

Magic commands switch a cell's language or trigger special behaviours within Databricks notebooks. They override the notebook's default language for that cell only, allowing you to mix Python, SQL, Markdown, and shell commands in a single notebook.

Command	Purpose	Example
%python	Execute cell as Python	%python df.show(5)
%sql	Execute cell as Spark SQL	%sql SELECT COUNT(*) FROM sales_db.orders
%scala	Execute cell as Scala	%scala spark.version
%r	Execute cell as R	%r summary(iris)
%md	Render cell as Markdown documentation	%md ## Section Heading
%fs	DBFS file system operations (ls, cp, rm)	%fs ls /databricks-datasets/
%run	Execute another notebook; shares SparkSession	%run /Shared/utils/common
%sh	Run shell commands on the driver node	%sh pip list

2.2 DBFS — Key Points

The Databricks File System (DBFS) is a POSIX-like abstraction over cloud object storage (ADLS Gen2, S3, GCS). Data physically lives in cloud storage — DBFS just provides convenient path aliases. Use dbfs:/FileStore for demos and small artifacts. For production, prefer direct cloud URIs (abfss://) or Unity Catalog Volumes.

2.3 Databricks Utilities (dbutils)

dbutils is the primary way to interact with the Databricks environment programmatically from notebooks — file operations, parameterised inputs, and secure secret retrieval.

Utility	Key Methods	Purpose
dbutils.fs	ls(), mkdirs(), cp(), mv(), rm()	Navigate and manipulate DBFS and cloud storage paths
dbutils.widgets	text(), dropdown(), get()	Parameterise notebooks with interactive inputs; retrieve values at runtime
dbutils.secrets	get(scope, key)	Retrieve secrets from Secret Scopes (Azure Key Vault backed). Output always REDACTED.

```
# Create a text widget with a default value
dbutils.widgets.text('run_date', '2025-09-07', 'Run Date')
run_date = dbutils.widgets.get('run_date')

# Retrieve a secret — always prints [REDACTED]
secret_value = dbutils.secrets.get(scope='ans_scope', key='appSecret')

# File system operations
dbutils.fs.ls('dbfs:/')                                  # list directory
dbutils.fs.mkdirs('dbfs:/tmp/checkpoints/my_job')      # create directory
dbutils.fs.cp('dbfs:/src/file.csv', 'dbfs:/dst/')      # copy file
```

Security Best Practice

NEVER hard-code credentials, secrets, or connection strings in notebook cells. Always use Azure Key Vault linked to a Databricks Secret Scope, then retrieve at runtime with dbutils.secrets.get(). Output is always REDACTED in logs — secrets are never visible in plain text.

3. PySpark Transformations & Spark SQL

3.1 Creating DataFrames

DataFrames are the primary data abstraction in Spark — distributed, immutable, tabular datasets with a named schema. All transformations on DataFrames are lazy and produce new DataFrames. Execution is triggered only by an Action.

```
# Inline data — quick creation for testing
my_data   = [(1, 'Alice', 30), (2, 'Bob', 40), (3, 'Charlie', 50)]
my_schema = 'ID INT, Name STRING, Marks INT'
df = spark.createDataFrame(my_data, my_schema)

# Explicit StructType schema — recommended for production
from pyspark.sql.types import StructType, StructField, IntegerType, StringType
schema = StructType([
    StructField('ID',    IntegerType(), True),
    StructField('Name',  StringType(),  True),
    StructField('Marks', IntegerType(), True),
])
df = spark.createDataFrame(my_data, schema)
```

3.2 Essential Transformations

PySpark provides a rich API for transforming DataFrames. All transformations are lazy — they build the computation DAG but do not execute until an Action is called. Always prefer built-in functions from pyspark.sql.functions over Python UDFs — built-in functions run natively in the JVM through Spark's code generation with no serialisation overhead.

Function	Purpose	Example
withColumn + split	Split string into array on delimiter	F.split(F.col('item_type'), ' ')
withColumn + lit	Add a constant column to every row	F.lit('processed')
withColumn + cast	Change column data type explicitly	F.col('amount').cast('double')
withColumn + date_format	Extract or format date/time parts	F.date_format(F.col('ts'), 'EEEE')
na.fill	Replace NULL/NaN values	df.na.fill({'amount':0, 'name':'Unknown'})
filter / where	Filter rows by condition (pushdown to source)	df.filter(F.col('Marks') > 50)
select	Project specific columns	df.select('ID', 'Name', F.col('Marks').alias('score'))
groupBy + agg	Group rows and compute aggregates	df.groupBy('country').agg(F.sum('sales'))
coalesce(n)	Reduce partition count — no shuffle	df.coalesce(4) — use before write()
repartition(n)	Increase/rebalance partitions — full shuffle	df.repartition(8, 'region')

```
from pyspark.sql import functions as F
from pyspark.sql.types import StringType

df = df.withColumn('item_type_list', F.split(F.col('item_type'), ' '))
df = df.withColumn('flag_column',    F.lit('processed'))
df = df.withColumn('item_visibility',F.col('item_visibility').cast(StringType()))
df = df.withColumn('day_of_week',    F.date_format(F.col('InvoiceDate'), 'EEEE'))
df = df.na.fill({'amount': 0})
df.display()
```

3.3 Spark SQL & Views

Spark SQL integrates DataFrame APIs with ANSI SQL queries. Register any DataFrame as a view and query it with SQL — the result is a new DataFrame. All APIs (SQL, DataFrame, Dataset) compile to the same Catalyst execution plan, so there is no performance difference between them.

View Type	Scope / Lifetime	Use Case
Permanent View	Stored in metastore — persists across sessions and clusters	Reusable business logic, shared data models, governed queries
Temporary View	Current SparkSession only — dropped when session ends	Intermediate results in a single notebook or job
Global Temp View	All notebooks on same cluster — dropped when cluster stops	Short-lived cross-notebook collaboration on the same cluster

```
# Register and query
df.createOrReplaceTempView('sales_data')
result = spark.sql('''
    SELECT country, SUM(amount) AS total_sales, COUNT(*) AS num_orders
    FROM   sales_data
    WHERE  year = 2025
    GROUP  BY country ORDER BY total_sales DESC LIMIT 10
''')

# Permanent view in Unity Catalog
# CREATE VIEW prod.sales.monthly_revenue AS SELECT ...

# Global temp view
df.createOrReplaceGlobalTempView('shared_data')
spark.sql('SELECT * FROM global_temp.shared_data').show()
```

3.4 Spark Reader API

The DataFrame Reader API (spark.read) is the standard way to ingest structured and semi-structured data from cloud storage. For production, always use explicit schemas rather than inferSchema — inferSchema triggers an extra full scan of the data, which is expensive at scale.

```
# Read CSV from ADLS Gen2
df = (spark.read
    .format('csv')
    .option('header', True)
    .option('inferSchema', True)   # avoid in production — use explicit schema
```

```
    .option('mode', 'PERMISSIVE')  # PERMISSIVE | DROPMALFORMED | FAILFAST
    .load('abfss://source@account.dfs.core.windows.net/data/'))
```

mode Option	Behaviour
PERMISSIVE	Keeps corrupt rows; places them in a _corrupt_record column. Default.
DROPMALFORMED	Silently drops corrupt/malformed rows. Risk of silent data loss.
FAILFAST	Throws an exception immediately on the first corrupt record. Good for strict validation.

3.5 User-Defined Functions (UDFs)

UDFs extend Spark with custom transformation logic. They should be used only when built-in functions cannot express the required logic — built-in functions are always faster because they run natively in the JVM without Python serialisation overhead.

UDF Type	Performance	When to Use
Built-in Spark functions	Fastest — JVM-native, Catalyst-optimised code generation	Always prefer over any UDF
Pandas (Vectorised) UDF	Fast — operates on pandas Series batches, fewer JVM crossings	Preferred Python UDF option for heavy transformations
Scala/Java UDF	Fast — runs in JVM with no serialisation penalty	Performance-critical logic when team is comfortable in Scala
Standard Python UDF	Slowest — row-by-row JVM↔Python serialisation overhead	Last resort for simple logic not expressible in built-ins

4. Delta Lake

4.1 What is Delta Lake?

Delta Lake is an open-source storage layer that adds database-grade reliability to data lakes. It is not a new file format — Delta = Parquet files + a Transaction Log (_delta_log). The transaction log is what gives Delta its superpowers: ACID transactions, time travel, schema enforcement, and unified batch/streaming on top of plain cloud object storage (ADLS Gen2, S3, GCS).

Delta Lake forms the storage backbone of the modern Lakehouse architecture — combining the scalability and cost-efficiency of a data lake with the reliability and governance of a data warehouse.

Feature	What It Does	Why It Matters
ACID Transactions	Every write is atomic — either fully committed or fully rolled back. No partial writes.	Safe concurrent reads/writes. No corrupt data from failed jobs.
Transaction Log	_delta_log directory of JSON files — one per commit, recording files added/removed, schema changes.	Enables time travel, snapshot reads, and audit history without scanning all files.
Schema Enforcement	Rejects writes that don't match the table's defined schema.	Prevents data corruption from upstream schema drift or bugs.
Schema Evolution	Add new columns incrementally without rewriting data (mergeSchema=true).	Handles evolving data sources gracefully.
Time Travel	Query or restore data at any previous version number or timestamp.	Rollback bad writes, auditing, ML experiment reproducibility.
Unified Batch+Streaming	Same Delta table used by batch and streaming jobs simultaneously.	Eliminates Lambda architecture complexity.
Deletion Vectors	Mark row-level changes externally instead of full-file rewrite.	Faster UPDATEs/DELETEs — avoids expensive file rewrites (DBR 8.0+).

4.2 Delta Table Physical Structure

Understanding what Delta Lake stores on disk is a common interview topic. Each Delta table directory contains Parquet data files and a _delta_log subdirectory containing the transaction log.

```
my_delta_table/
├── part-00001-abc.snappy.parquet   ← Columnar data files
├── part-00002-def.snappy.parquet
└── _delta_log/                     ← Transaction log
    ├── 00000000000000000000.json   ← Version 0: CREATE TABLE
    ├── 00000000000000000001.json   ← Version 1: INSERT
    ├── 00000000000000000002.json   ← Version 2: UPDATE (tombstones old, adds new)
    └── *.crc                       ← Integrity checksums

How reads work:
  Spark reads _delta_log → builds latest snapshot → identifies active Parquet files →
queries only those
```

4.3 Managed vs. External Delta Tables

The choice between managed and external tables is one of the most frequently asked Delta Lake interview topics. The key difference is data lifecycle ownership — who controls the physical data files.

Feature	Managed Table	External Table
Data location	Databricks-managed storage (metastore root or catalog/schema managed location)	User-specified cloud storage path — LOCATION clause required on CREATE
DROP TABLE behaviour	Deletes BOTH metadata AND data files (Unity Catalog: soft-delete, 7-day UNDROP window)	Deletes metadata ONLY — physical data files remain intact in cloud storage
Data lifecycle	Fully managed by Databricks — creation to deletion	User controls physical files independently of the metastore
Best for	Gold/curated datasets, full lifecycle managed by Databricks	Bronze/raw zones, independent data ownership, protection from accidental deletion

```
-- Managed table (no LOCATION — Databricks chooses storage)
CREATE TABLE sales_db.managed_tbl (ID INT, Name STRING, Marks INT) USING DELTA;

-- External table (explicit LOCATION — you control the data)
CREATE TABLE sales_db.external_tbl (ID INT, Name STRING, Marks INT)
USING DELTA
LOCATION 'abfss://my_container@account.dfs.core.windows.net/ext_table/';

-- Unity Catalog: restore soft-deleted managed table (within 7 days)
UNDROP TABLE sales_db.managed_tbl;
```

4.4 DML Operations

Delta Lake supports the full SQL DML suite with ACID guarantees. Each operation interacts with the transaction log in a specific way — understanding this helps you predict performance and storage behaviour.

INSERT

INSERT creates new Parquet files and appends 'add' actions to the transaction log. It does not prevent duplicates — use MERGE INTO for upsert/deduplication scenarios.

```
INSERT INTO sales_db.managed_tbl VALUES (1, 'Alice', 95), (2, 'Bob', 87);
```

UPDATE

UPDATE rewrites affected Parquet files and tombstones originals in the transaction log. With Deletion Vectors enabled (DBR 8.0+), only changed rows are written to a small file — avoiding full-file rewrites for small updates.

```
UPDATE sales_db.managed_tbl SET Marks = 100 WHERE ID = 1;
```

DELETE

DELETE rewrites files excluding deleted rows and tombstones the originals. Original files remain available for time travel until VACUUM removes them.

```
DELETE FROM sales_db.managed_tbl WHERE Marks < 0;
```

MERGE INTO — Upserts (Most Important)

MERGE INTO performs UPDATE + INSERT (and optionally DELETE) in a single ACID transaction. It is the gold standard for Change Data Capture (CDC) pipelines, upsert patterns, and deduplication. MERGE guarantees atomicity — either all changes succeed or none do.

```
MERGE INTO sales_db.managed_tbl AS target
USING       sales_db.updates    AS source
ON          target.ID = source.ID
WHEN MATCHED
    THEN UPDATE SET target.Name = source.Name, target.Marks = source.Marks
WHEN NOT MATCHED
    THEN INSERT (ID, Name, Marks) VALUES (source.ID, source.Name, source.Marks);
```

4.5 Time Travel & History

Every DML operation creates a new version in the transaction log. Delta's time travel feature lets you query any previous version — invaluable for auditing, debugging, ML reproducibility, and recovering from bad writes.

```
-- View full version history
DESCRIBE HISTORY sales_db.managed_tbl;

-- Query at a specific version
SELECT * FROM sales_db.managed_tbl VERSION AS OF 5;

-- Query at a specific timestamp
SELECT * FROM sales_db.managed_tbl TIMESTAMP AS OF '2025-09-06 10:30:00';

-- Restore entire table to a previous version
RESTORE TABLE sales_db.managed_tbl TO VERSION AS OF 2;
```

Time Travel Use Case	When to Use It
Rollback bad write	A corrupt INSERT/UPDATE happened → RESTORE TABLE to version before the bad write
Audit / compliance	Regulators need data as it was on a specific date → TIMESTAMP AS OF
ML reproducibility	Reproduce the exact training dataset used for a specific model run → VERSION AS OF
Debug pipeline issues	Compare data at two versions to identify what a transformation step changed

4.6 Optimization Techniques

Delta tables require periodic maintenance to maintain optimal query performance. Three operations are central: OPTIMIZE (file compaction), Z-ORDER BY (data-skipping indexing), and VACUUM (garbage collection).

OPTIMIZE — File Compaction

Frequent small writes (streaming micro-batches, many small INSERTs) produce hundreds of tiny Parquet files. The query planner must open metadata for every file — thousands of small files dramatically increase planning overhead and cloud storage API costs. OPTIMIZE coalesces small files into fewer, larger files (targeting ~1 GB each).

```
OPTIMIZE sales_db.managed_tbl;                          -- compact all files
OPTIMIZE sales_db.managed_tbl ZORDER BY (Name, ID);     -- compact + data-skipping
OPTIMIZE sales_db.managed_tbl WHERE date = '2025-09-01'; -- targeted compaction
```

Z-ORDER BY — Data Skipping

Z-ORDER BY physically reorganises data within Parquet files so rows with similar values in the Z-ORDER columns are co-located. Spark records min/max statistics per file — on queries with WHERE filters on Z-ORDER columns, it skips entire files that can't contain matching rows. Apply Z-ORDER to high-cardinality columns commonly used in WHERE clauses (customer_id, date, region). Do not apply to low-cardinality columns (boolean flags) — no benefit.

VACUUM — Garbage Collection

VACUUM permanently deletes tombstoned Parquet files older than the retention window (default 7 days / 168 hours). After VACUUM removes a file, time travel to any version referencing that file becomes permanently impossible. Never VACUUM below 7 days in production.

```
VACUUM sales_db.managed_tbl RETAIN 168 HOURS;   -- standard 7-day retention
VACUUM sales_db.managed_tbl RETAIN 168 HOURS DRY RUN;  -- preview what would be deleted
```

> ⚠ **VACUUM Warning**
> After VACUUM removes files, time travel to those versions is permanently impossible.
> Always retain at least 168 hours (7 days) in production. Never set to 0 hours in production.
> Run VACUUM regularly — weekly, or after large DELETE/UPDATE/MERGE operations.

4.7 Deep Clone vs. Shallow Clone

Delta Lake supports two types of table cloning — Deep and Shallow — serving different purposes. Both create a new table with an independent transaction log but differ in whether data files are physically copied.

Feature	Deep Clone	Shallow Clone
Data copied?	YES — full independent copy of metadata + all Parquet data files	NO — metadata only; new table references source data files
Independence	Fully independent — source changes not reflected; VACUUM of source safe	Dependent — breaks if source files are removed by VACUUM
Storage cost	Higher (duplicates all data)	Near-zero (no data duplication)

Creation speed	Slower (proportional to data size)	Near-instant regardless of data size
Best for	Production backups, cloud migrations, independent test environments	Quick dev/test without duplicating large datasets

```
CREATE TABLE sales_db.tbl_deep_clone    DEEP    CLONE sales_db.managed_tbl;
CREATE TABLE sales_db.tbl_shallow_clone SHALLOW CLONE sales_db.managed_tbl;
```

5. Unity Catalog

5.1 What is Unity Catalog?

Unity Catalog (UC) is Databricks' centralised governance layer for all data and AI assets. Before UC, each workspace had its own isolated Hive Metastore and user management — creating governance silos where policies, permissions, and metadata were duplicated and inconsistent across workspaces.

UC solves this with a single metastore per cloud region that all workspaces attach to. Policies are defined once and enforced everywhere. One audit log, one lineage graph, one catalog for all workspaces attached to the same metastore.

Feature	Without Unity Catalog	With Unity Catalog
Governance scope	Per-workspace isolation — policies not shared between workspaces	Account-wide — one metastore governs all attached workspaces
User management	Per-workspace — must be replicated manually across workspaces	Centralised — users, groups, permissions defined once
Data discovery	Siloed — data in workspace A not visible from workspace B	Unified catalog — all data discoverable from any workspace
Audit logging	Per-workspace, not centralised	Single audit log: system.access.audit for all workspaces
Access control	Hive-style per-workspace ACLs	ANSI SQL GRANT/REVOKE at catalog, schema, table, column level

5.2 The Three-Level Namespace

All Unity Catalog objects are organised in a three-level hierarchy. Every object is referenced as catalog.schema.object_name. This consistent naming makes it easy to scope access control grants and understand data ownership.

Level	Object	Description
Top	Metastore	One per cloud region per account. All workspaces in the region attach to it.
Level 1	Catalog	Environment boundary (prod, dev, qa) or business domain (finance, sales, hr).
Level 2	Schema (Database)	Project or team grouping within a catalog.
Level 3	Table	Managed or external tabular data (Delta, Parquet, CSV, JSON, ORC, Avro).
Level 3	View	Saved SQL query over tables or other views. No data stored.
Level 3	Volume	Governed file access (managed or external) for non-tabular data.
Level 3	Function / Model	Registered UDFs and MLflow AI models — governed as securables.

5.3 Managed vs. External — Tables & Volumes (UC)

Feature	Managed	External
Storage	Metastore root / catalog-schema managed location	User-specified LOCATION clause (External Location required)
DROP behaviour	Soft-deletes data + metadata; UNDROP within 7 days	Removes metadata only — data intact in external storage
Best for	Gold/curated datasets, Databricks-managed lifecycle	Bronze/raw zones, retain data ownership, external storage governance

5.4 Location Priority for Managed Objects

When creating a managed table or volume without specifying a LOCATION, Databricks resolves where to store data using the following priority order. The highest applicable level wins:

Priority	Level	Condition
1 (Highest)	Table LOCATION	Explicit LOCATION clause on the CREATE TABLE / VOLUME statement
2	Schema LOCATION	No table-level LOCATION; the parent schema has a managed location
3	Catalog LOCATION	No table or schema-level LOCATION; the parent catalog has a managed location
4 (Default)	Metastore root storage	No LOCATION at any level — data stored at the metastore's default root

5.5 External Locations & Storage Credentials

For data in external cloud storage, Unity Catalog uses a two-layer model: a Storage Credential (cloud identity) and an External Location (URI mapping with access grants).

```
-- Create a Storage Credential (Azure Managed Identity)
CREATE STORAGE CREDENTIAL anch_creds
WITH AZURE_MANAGED_IDENTITY
COMMENT 'Bound to Azure Databricks Access Connector';

-- Create an External Location
CREATE EXTERNAL LOCATION my_external_loc
URL 'abfss://my_container@storagemodDB.dfs.core.windows.net/'
WITH CREDENTIAL anch_creds;

-- Grant access
GRANT READ FILES  ON EXTERNAL LOCATION my_external_loc TO `data_ingestion_group`;
GRANT WRITE FILES ON EXTERNAL LOCATION my_external_loc TO `data_engineers`;
```

5.6 Granting Permissions — Key Rules

Unity Catalog uses ANSI SQL GRANT/REVOKE syntax. The most important rule: always grant USAGE at the catalog and schema level before granting object-level privileges — without USAGE, object-level grants have no effect.

```
-- Always USAGE first, then object-level
GRANT USAGE  ON CATALOG prod                    TO `data_engineers`;
GRANT USAGE  ON SCHEMA  prod.sales              TO `data_engineers`;
GRANT SELECT ON ALL TABLES IN SCHEMA prod.sales TO `bi_readers`;
GRANT MODIFY ON TABLE   prod.sales.orders       TO `data_engineers`;

-- Volumes
GRANT READ FILES ON EXTERNAL LOCATION my_loc TO `ingestion_svc`;
```

5.7 Fine-Grained Access Control — Row Filters & Column Masking

Unity Catalog provides two powerful mechanisms for fine-grained data access control at the row and column level — both transparent to the query author.

Row Filters

Row Filters apply an invisible SQL WHERE clause per user or group. Users querying the table see only rows matching the filter predicate — no changes needed to their queries. Essential for multi-tenant architectures and regional data isolation.

```
-- Create a row filter function
CREATE OR REPLACE FUNCTION my_catalog.security.region_filter(region_col STRING)
    RETURN IS_ACCOUNT_GROUP_MEMBER('global_admin') OR region_col =
current_user_region();

-- Apply to table — filter enforced on every query automatically
ALTER TABLE my_catalog.silver.sales SET ROW FILTER my_catalog.security.region_filter ON
(region);
```

Column Masking

Column Masking transforms column values at query time based on the caller's identity. Different user groups see different representations of the same column — full value, masked, or NULL. Essential for PII protection.

```
-- Create a masking function
CREATE OR REPLACE FUNCTION my_catalog.security.mask_email(email STRING)
    RETURN CASE
        WHEN IS_ACCOUNT_GROUP_MEMBER('data_engineers') THEN email
        WHEN IS_ACCOUNT_GROUP_MEMBER('analysts')       THEN
CONCAT(LEFT(email,2),'****@****.com')
        ELSE NULL
    END;

-- Apply masking policy to a column
ALTER TABLE my_catalog.silver.customers ALTER COLUMN email SET MASK
my_catalog.security.mask_email;
```

5.8 Governed Tags & Data Lineage

Governed Tags

Tags are key-value metadata labels applied to any UC object (catalog, schema, table, column, volume). They are searchable, drive attribute-based access policies, and support compliance classification (PII, GDPR, HIPAA).

```
ALTER TABLE my_catalog.silver.customers SET TAGS ('pii' = 'true', 'domain' = 'crm');
ALTER TABLE my_catalog.silver.customers ALTER COLUMN email SET TAGS ('pii_type' =
'email');
SELECT * FROM system.information_schema.table_tags WHERE tag_name = 'pii' AND tag_value
= 'true';
```

Data Lineage

Unity Catalog automatically captures table-level and column-level lineage across SQL, Python, and pipeline transformations. Lineage is queryable via system tables and visualised in the Data Explorer as a lineage graph.

```
-- Query column-level lineage
SELECT source_table_full_name, source_column_name, target_table_full_name,
target_column_name
FROM   system.access.column_lineage
WHERE  target_table_full_name = 'prod.gold.customer_kpis';
```

6. Auto Loader & Structured Streaming

6.1 Structured Streaming — Core Concepts

Structured Streaming is Spark's high-level API for continuous stream processing. It treats streaming data as an unbounded DataFrame — an infinite table that continuously grows as new data arrives. The same DataFrame transformations used for batch data work identically for streaming, with Spark managing incremental processing, fault tolerance, and exactly-once semantics.

Concept	Description
Unbounded DataFrame	An infinite growing table. readStream returns a streaming DataFrame — never materialised all at once.
Trigger	How often the streaming job processes new data. processingTime='5 seconds' or availableNow=True (one-shot).
Checkpoint Location	Stores query progress (offsets, committed batches) for fault tolerance and exactly-once processing.
Output Mode — append	Only new rows since last trigger. Default for non-aggregate streams.
Output Mode — complete	Full aggregated result on every trigger. For aggregation queries.
Output Mode — update	Only rows that changed since last trigger. For aggregations where only deltas matter.

6.2 Auto Loader

Auto Loader is Databricks' incremental ingestion service built on the cloudFiles streaming source. It detects and processes only NEW files arriving in cloud storage — it never re-processes already-ingested files. It uses a RocksDB-backed metadata store to track processed files, guarantees exactly-once delivery, and supports automatic schema inference and evolution.

Parameter	Required?	Purpose
cloudFiles.format	Yes	Underlying file format: parquet, csv, json, avro, orc, etc.
cloudFiles.schemaLocation	Yes	Stores schema inference/evolution state. Must be a durable cloud path.
checkpointLocation	Yes	Stores streaming progress. Enables exactly-once delivery and restart recovery.
trigger(processingTime=...)	Recommended	Micro-batch interval. Use availableNow=True for one-shot batch-style ingestion.

```
# Auto Loader — complete production pattern
df_stream = (spark.readStream
    .format('cloudFiles')
    .option('cloudFiles.format', 'parquet')
    .option('cloudFiles.schemaLocation',
            'abfss://dest@account.dfs.core.windows.net/checkpoints/_schemas')
    .load('abfss://source@account.dfs.core.windows.net/raw/'))
```

```
query = (df_stream.writeStream
    .format('delta')
    .option('checkpointLocation',
            'abfss://dest@account.dfs.core.windows.net/checkpoints/_data')
    .option('mergeSchema', 'true')       # allow schema evolution
    .trigger(processingTime='5 seconds')
    .outputMode('append')
    .start('abfss://dest@account.dfs.core.windows.net/silver/sales/'))

# One-shot batch mode — process all available files then stop
# .trigger(availableNow=True)

# Always stop after tests
# query.stop()
```

Auto Loader Best Practices

Always store checkpointLocation and schemaLocation on durable cloud storage (ADLS/S3) — never on local driver storage.

Use availableNow=True for scheduled batch-style ingestion (processes all backlog, then stops cleanly).

Always stop streaming queries after notebook tests — running queries consume cluster compute continuously.

7. Databricks Workflows

7.1 What are Workflows?

Databricks Workflows is the built-in job orchestration engine for creating, scheduling, and managing multi-task data pipelines. Rather than running notebooks manually, Workflows lets you define a complete pipeline as a Directed Acyclic Graph (DAG) of tasks with dependencies, scheduling, retries, parameter passing, and monitoring — all in one managed service.

Concept	Description
Tasks	Discrete pipeline units: notebook, Python script, JAR, SQL task, DLT pipeline, dbt project.
DAG Dependencies	Tasks declare dependencies — downstream tasks run only after upstream tasks complete successfully.
Job Cluster	Recommended for production — fresh, isolated, auto-terminates after the run. Cheaper than interactive.
Interactive Cluster	Shared, always-on — use for development only, not production jobs.
Parameters	Key-value pairs passed to notebooks (via widgets) or scripts (base parameters).
Retries & Timeouts	Configure max retries, retry delay, and per-task timeouts for fault tolerance.
Repair & Re-run	Re-run only failed/skipped tasks — preserves successful task outputs, saves recovery time.
For Each Task	Dynamically iterates a sub-pipeline over a list (regions, dates, files) in parallel or sequentially.
Notifications	Email or webhook alerts on job/task success, failure, or retry.

Job Cluster vs Interactive Cluster

Always use job clusters for production workflows: they start fresh for each run (no leftover state), are isolated from other jobs, auto-terminate when the job finishes (saving cost), and are significantly cheaper per-DBU than interactive clusters.

Interactive clusters are for development only — they are shared, always-on, and expensive to leave running.

8. Lakeflow & Modern Data Engineering Features

8.1 Lakeflow Overview

Lakeflow is Databricks' next-generation data engineering platform that unifies ingestion, transformation, and orchestration under one brand. It brings together Lakeflow Connect (managed connectors), Lakeflow Declarative Pipelines (next-gen DLT), AUTO CDC (automated change data capture), and Lakeflow Designer (visual pipeline canvas) into a cohesive experience.

Component	Purpose	Key Benefit
Lakeflow Connect	Managed no-code connectors to 50+ sources (Salesforce, SQL Server, MySQL, ServiceNow, etc.)	No custom ingestion code — connectors maintained and scaled by Databricks
Lakeflow Declarative Pipelines	Next-gen DLT: declare transformations in SQL/Python; Databricks handles execution, retries, quality, ordering	Focus on transformation logic, not orchestration — automatic dependency resolution
AUTO CDC	Automated Change Data Capture within pipelines — handles MERGE, deduplication, event ordering automatically	Eliminates complex manual MERGE CDC logic — declare source, keys, and sequence column
Lakeflow Designer	Visual drag-and-drop pipeline canvas	Build pipelines without writing code — generates DLT definitions under the hood

8.2 Lakeflow Declarative Pipelines (DLT)

In a Lakeflow Declarative Pipeline, you declare transformation logic using SQL (LIVE keyword) or Python (@dlt.table). The pipeline engine automatically determines execution order, handles incremental processing, manages checkpointing, retries failures, enforces data quality EXPECT constraints, and publishes tables to Unity Catalog.

```
-- Bronze: raw streaming ingestion from Auto Loader
CREATE OR REFRESH STREAMING LIVE TABLE bronze_orders
COMMENT 'Raw orders from source — append only'
AS SELECT *, current_timestamp() AS _ingestion_time
   FROM   cloud_files('abfss://raw@account.dfs.core.windows.net/orders/', 'json');

-- Silver: validated with data quality expectations
CREATE OR REFRESH STREAMING LIVE TABLE silver_orders (
    CONSTRAINT valid_order_id  EXPECT (order_id IS NOT NULL) ON VIOLATION DROP ROW,
    CONSTRAINT positive_amount EXPECT (amount > 0)           ON VIOLATION DROP ROW
)
AS SELECT order_id, customer_id, CAST(amount AS DECIMAL(18,2)) AS amount, status
   FROM   STREAM(LIVE.bronze_orders);

-- Gold: aggregated business metrics
CREATE OR REFRESH LIVE TABLE gold_daily_revenue
AS SELECT order_date, SUM(amount) AS total_revenue, COUNT(*) AS order_count
   FROM   LIVE.silver_orders WHERE status = 'CLOSED' GROUP BY order_date;
```

8.3 AUTO CDC — Automated Change Data Capture

AUTO CDC (APPLY CHANGES INTO) is a first-class feature in Lakeflow Declarative Pipelines that automates CDC pipeline development. You declare the source, the keys identifying unique rows, and the sequence column for ordering. Lakeflow automatically applies changes in the correct order, handles late-arriving records, deduplicates events, and maintains the correct final state in the target Delta table — no manual MERGE code required.

AUTO CDC Feature	Description
Automatic MERGE handling	Translates INSERT/UPDATE/DELETE events into correctly ordered MERGE operations automatically
Out-of-order event handling	Uses sequence column to apply changes in logical order even when events arrive out of sequence
SCD Type 1	Overwrites existing row with latest values — no history retained (default)
SCD Type 2	Keeps full history — adds effective_from/effective_to dates and is_current flag for each change

```
-- AUTO CDC: APPLY CHANGES INTO
CREATE OR REFRESH STREAMING LIVE TABLE customers_silver
(customer_id BIGINT, name STRING, email STRING, updated_at TIMESTAMP);

APPLY CHANGES INTO LIVE.customers_silver
FROM   STREAM(LIVE.customers_bronze)
KEYS   (customer_id)                    -- primary key
APPLY  AS DELETE WHEN operation = 'DELETE'
SEQUENCE BY updated_at                  -- ordering column
STORED AS SCD TYPE 1;                   -- or SCD TYPE 2 for full history
```

8.4 Delta Lake Storage Enhancements

Automatic Liquid Clustering

Liquid Clustering is a next-generation replacement for Hive-style partitioning. Traditional partitioning creates directory-per-value structures — high-cardinality columns create millions of tiny files, and partition columns are hard to change later. Liquid Clustering uses flexible adaptive data layout, avoids the small-file problem, and allows clustering columns to be changed anytime with ALTER TABLE — without rewriting existing data.

```
-- Create table with Liquid Clustering
CREATE TABLE my_catalog.silver.events CLUSTER BY (event_date, user_id)
AS SELECT * FROM bronze.raw_events;

-- Add to existing table
ALTER TABLE my_catalog.silver.events CLUSTER BY (event_date, user_id);

-- Change clustering columns as query patterns evolve
ALTER TABLE my_catalog.silver.events CLUSTER BY (region, event_date);
```

Delta UniForm — Universal Format

Delta UniForm automatically maintains Iceberg (and Hudi) metadata alongside the Delta transaction log — all pointing to the same underlying Parquet files. This allows a single Delta table to be read by Iceberg-compatible engines (Presto, Trino, AWS Athena, Snowflake) without data duplication or format conversion.

```
-- Enable UniForm on a table
ALTER TABLE my_catalog.silver.transactions
SET TBLPROPERTIES ('delta.universalFormat.enabledFormats' = 'iceberg');
```

Optimized Writes & Auto Compaction

Optimized Writes reshuffles data before writing to produce right-sized Parquet files at write time — preventing the small-files problem from forming. Auto Compaction runs a lightweight background OPTIMIZE after writes when too many small files accumulate. Together they keep tables query-efficient without manual OPTIMIZE runs.

```
-- Enable on a table (persistent)
ALTER TABLE my_catalog.silver.events
SET TBLPROPERTIES (
    'delta.autoOptimize.optimizeWrite' = 'true',
    'delta.autoOptimize.autoCompact'   = 'true'
);
```

8.5 Databricks Asset Bundles (DABs)

Databricks Asset Bundles (DABs) is the official Infrastructure-as-Code (IaC) and CI/CD framework for Databricks. Define all resources — jobs, pipelines, clusters, permissions — as YAML configuration files in databricks.yml, commit to Git, and deploy via the Databricks CLI or CI/CD platforms (GitHub Actions, Azure DevOps, GitLab CI). This enables GitOps: every production change is version-controlled, reviewed, and deployed automatically.

DABs Concept	Description
databricks.yml	YAML config file defining all Databricks resources — jobs, pipelines, clusters, UC schemas, permissions.
Targets (environments)	Single bundle with per-target overrides: dev, staging, prod — one command deploys to any environment.
CLI commands	databricks bundle deploy / validate / run / destroy — full lifecycle management.
CI/CD integration	Official GitHub Actions and Azure DevOps extensions for automated deployment pipelines.

8.6 Data Federation (Lakehouse Federation)

Data Federation lets you query external databases (MySQL, PostgreSQL, SQL Server, Snowflake, Redshift, BigQuery) directly from Databricks using SQL, without physically moving or copying data. Create a Connection and Foreign Catalog in Unity Catalog — external tables appear alongside Delta tables, governed by the same GRANT/REVOKE permissions.

```
-- Create a Connection to external database
CREATE CONNECTION my_postgres_conn TYPE POSTGRESQL
OPTIONS (host 'postgres.mycompany.com', port '5432',
         user secret('scope','pg_user'), password secret('scope','pg_pass'));

-- Create Foreign Catalog pointing to external DB
CREATE FOREIGN CATALOG postgres_prod USING CONNECTION my_postgres_conn
OPTIONS (database 'production');
```

```
-- Query external + Delta tables together — no data movement
SELECT c.customer_id, SUM(o.amount) AS lifetime_value
FROM   postgres_prod.public.customers c
JOIN   my_catalog.gold.orders         o ON c.customer_id = o.customer_id
GROUP  BY c.customer_id;
```

9. Medallion Architecture

9.1 Overview

The Medallion Architecture is a layered data design pattern native to the Databricks Lakehouse. It organises data into three progressive quality zones — Bronze, Silver, and Gold — each serving a distinct purpose and improving data quality, reliability, and query performance at each stage. It is one of the most commonly discussed architecture patterns in Databricks interviews.

Layer	Data State	Key Operations	Storage & Governance
Bronze	Raw, unprocessed. Append-only. May have duplicates, schema errors, nulls. Full fidelity.	Auto Loader ingestion. Add ingestion timestamp and source metadata. Schema-on-read.	External Delta tables. Append-only writes. Retain ALL raw data — never delete Bronze.
Silver	Cleaned, deduplicated, type-cast, validated. Business rules applied. Schema enforced.	MERGE INTO for deduplication. NULL handling. Type casting. Joining reference data.	Managed or external tables. MERGE for upsert. Data engineering team owns MODIFY.
Gold	Aggregated, business-ready. Pre-computed KPIs and metrics. Optimised for consumption.	GROUP BY aggregations. OPTIMIZE + Z-ORDER for query speed. SLA enforcement.	Managed tables. OPTIMIZE + Z-ORDER. BI readers have SELECT only. Tightly governed.

Key Interview Point

The Medallion Architecture makes pipelines modular, auditable, and recoverable. If a Silver transformation has a bug, you fix and re-run it from preserved Bronze data — no re-ingestion from source required. Each layer maps to a Unity Catalog schema with appropriate permissions: Bronze = raw External tables; Silver = MERGE-based managed tables; Gold = OPTIMIZE+Z-ORDER managed tables for BI.

10. Summary & Quick Revision Cheat Sheet

10.1 Key Concept Comparisons

Concept A	vs	Concept B	Key Difference
Managed Table	vs	External Table	DROP deletes data (+ metadata) vs DROP deletes metadata ONLY
Deep Clone	vs	Shallow Clone	Full independent data copy vs metadata only (references source files)
OPTIMIZE	vs	Z-ORDER	Compacts small files vs co-locates data for skipping (can be combined)
coalesce(n)	vs	repartition(n)	Reduces partitions without shuffle vs full shuffle (can increase)
Temp View	vs	Global Temp View	Current session only vs all notebooks on same cluster
Narrow Transform	vs	Wide Transform	No shuffle (filter, map) vs shuffle (groupBy, join, sort)
Auto Loader	vs	spark.read	Incremental (new files only, streaming) vs one-time batch read of all files
Liquid Clustering	vs	Hive Partitioning	Flexible adaptive clustering (changeable) vs rigid directory-per-value
Delta UniForm	vs	Delta Sharing	Same table as Iceberg (no copy) vs cross-org data sharing protocol
Row Filter	vs	Column Mask	Hide rows per user/group vs transform column values per user/group
Job Cluster	vs	Interactive Cluster	Fresh, isolated, auto-terminates (production) vs shared, always-on (dev)
SCD Type 1	vs	SCD Type 2	Overwrite current row (no history) vs full change history with effective dates
INSERT	vs	MERGE INTO	Append only, no dedup vs upsert with full MATCHED/NOT MATCHED logic
OPTIMIZE	vs	Auto Compaction	Manual compaction on demand vs automatic background compaction after writes
processingTime trigger	vs	availableNow trigger	Continuous micro-batch (runs indefinitely) vs one-shot batch then stops

10.2 Essential SQL Commands

```
-- ── DELTA TABLE CREATION ──────────────────────────────────
CREATE TABLE db.managed_tbl (ID INT, Name STRING, Marks INT) USING DELTA;
CREATE TABLE db.external_tbl (...) USING DELTA LOCATION 'abfss://...';
UNDROP TABLE db.managed_tbl;   -- UC only, within 7 days

-- ── DML ──────────────────────────────────────────────────
```

```
INSERT INTO db.tbl VALUES (1, 'Alice', 95);
UPDATE db.tbl SET Marks = 100 WHERE ID = 1;
DELETE FROM db.tbl WHERE Marks < 0;

MERGE INTO target t USING source s ON t.ID = s.ID
WHEN MATCHED     THEN UPDATE SET t.Name = s.Name, t.Marks = s.Marks
WHEN NOT MATCHED THEN INSERT (ID, Name, Marks) VALUES (s.ID, s.Name, s.Marks);

-- ── TIME TRAVEL ──────────────────────────────────────────────
DESCRIBE HISTORY db.tbl;
SELECT * FROM db.tbl VERSION AS OF 5;
SELECT * FROM db.tbl TIMESTAMP AS OF '2025-09-06 10:30:00';
RESTORE TABLE db.tbl TO VERSION AS OF 2;

-- ── OPTIMISATION ─────────────────────────────────────────────
OPTIMIZE db.tbl;
OPTIMIZE db.tbl ZORDER BY (col1, col2);
VACUUM   db.tbl RETAIN 168 HOURS;
VACUUM   db.tbl RETAIN 168 HOURS DRY RUN;

-- ── CLONING ──────────────────────────────────────────────────
CREATE TABLE clone_tbl DEEP    CLONE db.tbl;
CREATE TABLE clone_tbl SHALLOW CLONE db.tbl;

-- ── UNITY CATALOG ────────────────────────────────────────────
CREATE CATALOG prod MANAGED LOCATION 'abfss://...';
CREATE SCHEMA  prod.sales;
USE CATALOG prod;  USE SCHEMA sales;

GRANT USAGE  ON CATALOG prod                   TO `engineers`;
GRANT USAGE  ON SCHEMA  prod.sales             TO `engineers`;
GRANT SELECT ON ALL TABLES IN SCHEMA prod.sales TO `bi_readers`;

-- ── LIQUID CLUSTERING ────────────────────────────────────────
ALTER TABLE my_catalog.silver.events CLUSTER BY (event_date, user_id);

-- ── TAGS ─────────────────────────────────────────────────────
ALTER TABLE cat.schema.tbl SET TAGS ('pii' = 'true', 'domain' = 'crm');

-- ── AUDIT LOG ────────────────────────────────────────────────
SELECT * FROM system.access.audit WHERE event_time >= current_timestamp() - INTERVAL 1
DAY;
```

10.3 Top Topics by Interview Frequency

Priority	Topic	Most Likely Questions
★★★	Delta Lake — MERGE INTO	Full syntax, when to use vs INSERT, atomicity, CDC use case
★★★	Managed vs External Tables	DROP behaviour difference, when to use each, UNDROP
★★★	Time Travel	VERSION AS OF, TIMESTAMP AS OF, RESTORE, VACUUM impact
★★★	Unity Catalog — Object Model	3-level namespace, metastore, location priority
★★★	Lazy Evaluation	What it is, why it matters, Catalyst optimizer, transformations vs actions
★★	Auto Loader	cloudFiles, checkpointLocation, schemaLocation, exactly-once

★★	Medallion Architecture	Bronze/Silver/Gold definition, what goes in each layer
★★	OPTIMIZE / Z-ORDER / VACUUM	Purpose, when to run, VACUUM risk
★★	Narrow vs Wide Transformations	Shuffle vs no shuffle, stage boundaries, performance impact
★★	Row Filters & Column Masking	Difference, use cases, how applied transparently
★	Liquid Clustering vs Partitioning	When to use, flexibility advantage, no-rewrite column change
★	Delta UniForm	Iceberg compatibility, no data copy, use case
★	AUTO CDC (APPLY CHANGES INTO)	SCD Type 1 vs 2, SEQUENCE BY, how it replaces manual MERGE
★	Workflows — Job vs Interactive Cluster	Why job clusters for production, Repair Run concept

Final Interview Tips

1. Always explain the WHY — interviewers want to understand your reasoning, not just syntax recall.
2. MERGE INTO is the most tested Delta operation — know the full syntax and when to use it over INSERT.
3. For Unity Catalog, always mention the USAGE grant requirement and the 4-level location priority.
4. For Delta performance, cover OPTIMIZE (compaction), Z-ORDER (data skipping), VACUUM (cleanup) as a trio.
5. For streaming, mention checkpointLocation and exactly-once semantics — they signal production experience.
6. Know the Medallion Architecture cold — Bronze/Silver/Gold, what goes in each, and which Delta operations apply.
7. Liquid Clustering, Delta UniForm, and AUTO CDC are hot 2025 interview topics — a brief, confident explanation stands out.

— Good Luck with Your Interview! —